Tarea: Modelos No Lineales y SVM

Ciencia de Datos — Universidad Católica del Norte

Objetivos de esta tarea

- Construir desde cero un clasificador lineal binario y entrenarlo con descenso por gradiente estocástico (SGD) sobre la pérdida *hinge*, usando solo la biblioteca estándar de Python.
- Entender que la elección de características determina lo que el modelo puede aprender: encontrar las reseñas peor clasificadas y las características que las hundieron.
- Implementar en `numpy` un clasificador de margen suave y observar qué controla exactamente el parámetro C: cuántos ejemplos terminan siendo vectores de soporte y cómo se reparte el error entre entrenamiento y validación.
- Escribir un kernel como una función de dos vectores y comprobar, con el caso polinomial de grado 2, que su valor coincide con el producto punto en el espacio transformado: el truco del kernel en concreto.
- Comparar un modelo no lineal con el clasificador lineal de las partes anteriores sobre las mismas dos características, y medir cuánto cambia el error al curvar la frontera.

Contexto

Esta tarea acompaña la sesión de **modelos no lineales** del curso: el recorrido que va del predictor lineal a las características no lineales, y de ahí al margen máximo y al truco del kernel. Un clasificador lineal binario predice

$$f_{\mathbf{w}}(x) = \text{signo}\,(\mathbf{w} \cdot \phi(x)), \qquad \phi : \mathcal{X} \to \mathbb{R}^d,$$

donde ϕ es el **extractor de características**: la función que convierte un objeto del mundo (aquí, el texto de una reseña) en un vector de números. El vector $\mathbf{w}$ se aprende de los datos; ϕ lo diseña quien construye el sistema. En los Problemas 1 y 2 vas a comprobar, en concreto, que *la mayor parte del trabajo de modelamiento está en* ϕ; en el Problema 3 vas a ver qué agrega, sobre esa misma base, cambiar la función que se minimiza y la forma de la frontera.

La tarea es clasificar reseñas de cine como positivas ($y = +1$) o negativas ($y = -1$). La representación de partida es la **bolsa de palabras**: $\phi(x)$ asigna a cada palabra la cantidad de veces que aparece en la reseña. Una palabra es, simplemente, una cadena delimitada por espacios en blanco; no normalizamos mayúsculas, así que `Buena` y `buena` son características distintas.

Definimos el **puntaje** $\mathbf{w} \cdot \phi(x)$ y el **margen** $(\mathbf{w} \cdot \phi(x))\, y$. El margen es positivo cuando la predicción acierta y su magnitud indica cuán seguro estaba el modelo. La pérdida *hinge* es

$$\text{Pérd}_{\text{hinge}}(x, y, \mathbf{w}) = \text{máx}\{0,\ 1 - (\mathbf{w} \cdot \phi(x))\, y\}.$$

Por qué las características de palabras no dependen del idioma

El corpus de esta tarea son reseñas en inglés (es un conjunto de referencia clásico, con etiquetas de calidad). Eso no le quita generalidad al ejercicio: el extractor de características de palabras **no sabe inglés**. Lo único que hace es cortar la cadena por los espacios en blanco y contar cadenas repetidas. No consulta un diccionario, ni una lista de palabras positivas, ni reglas gramaticales de ningún idioma: cada cadena distinta se vuelve una coordenada del vector, y el peso de esa coordenada lo decide el

descenso por gradiente a partir de los datos etiquetados. Si mañana le entregas reseñas en español, el mismo código produce, por ejemplo,

$$\phi(\texttt{la película es muy buena}) = \{\texttt{la}{:}1,\ \texttt{película}{:}1,\ \texttt{es}{:}1,\ \texttt{muy}{:}1,\ \texttt{buena}{:}1\},$$

$$\phi(\texttt{no es una buena película}) = \{\texttt{no}{:}1,\ \texttt{es}{:}1,\ \texttt{una}{:}1,\ \texttt{buena}{:}1,\ \texttt{película}{:}1\},$$

y con suficientes ejemplos etiquetados aprendería, sin que nadie se lo diga, que `buena` pesa positivo y `no` pesa negativo. Fíjate en que las dos reseñas del ejemplo tienen sentimiento opuesto y comparten tres características: el idioma cambia, pero el problema de fondo es el mismo en los dos casos. La bolsa de palabras pierde el orden y, con él, la negación: `not good` y `good` comparten la característica que decide. Ningún vector de pesos arregla eso, porque la información ya se perdió al construir ϕ.

Hay dos lugares donde el idioma sí importa, y los dos son parte de la tarea:

- **Idiomas que no separan palabras con espacios** (chino, japonés, tailandés): ahí cortar por espacios entrega una sola "palabra" gigante y la bolsa de palabras deja de funcionar.
- **Morfología rica**: en español `aburrida`, `aburrido` y `aburridísima` son tres características sin relación entre sí para el modelo, aunque comparten la raíz.

En ambos casos ayuda cambiar ϕ por n-gramas de caracteres (todas las subcadenas de largo n, ignorando los espacios), que es justo lo que implementarás en el Problema 1c: las tres formas de `aburrid-` comparten el 6-grama `aburri`, así que el modelo puede reutilizar lo aprendido.

Archivos

- `submission.py`: el único archivo que entregas. Trae la sección de respuestas del comienzo, con la reseña del Problema 2d y los números del Problema 3c, y más abajo las doce funciones que debes programar.
- `util.py`: vectores dispersos, lectura del corpus, la representación numérica del Problema 3 y utilidades de análisis. Ya está completo.
- `grader.py`: corrector automático. Corre sin conexión a internet con `python grader.py`.
- `requirements.txt`: la única dependencia externa es `numpy`, que usa el Problema 3. Instálala con `pip install -r requirements.txt`.
- `datos/resenas.train` y `datos/resenas.dev`: 3.554 reseñas de entrenamiento y 3.554 de validación, una por línea, con formato `<etiqueta> <texto>`.

El corrector que viene con la tarea comprueba 60 de los 100 puntos: llegar a 60 de 60 es condición necesaria para el puntaje total, no suficiente. La nota la pone el corrector del curso, que corre esas mismas pruebas y agrega los otros 40: pruebas ocultas sobre las mismas funciones y la revisión de la sección de respuestas. Todo el puntaje de la tarea se reparte solo.

Qué entregar

Un solo archivo: `submission.py`, con las doce funciones completadas y todas las constantes de la sección de respuestas completadas.
No entregues nada más: ni la carpeta `datos/`, ni `util.py`, ni los archivos que genera el programa (`pesos.txt`, `analisis-error.txt`). Antes de entregar, verifica que `python grader.py` corre sin errores y llega a 60 de 60.

Problema 1: Clasificador de sentimientos [48 puntos]

Todo lo de este problema va en `submission.py`. Recuerda: aquí solo biblioteca estándar (nada de `numpy`, `scikit-learn` ni `nltk`), y conviene usar `dot_product` e `increment` de `util.py` en vez de manipular los diccionarios a mano.

(a) Características de palabras [8 puntos]

Implementa `extract_word_features(x)`, que recibe el texto de una reseña y devuelve $\phi(x)$ como diccionario: cada palabra (cadena separada por espacios en blanco) apuntando a la cantidad de veces que aparece.

```
>>> extract_word_features("me gusta y me gusta")
{'me': 2, 'gusta': 2, 'y': 1}
```

(b) Entrenamiento con SGD [30 puntos]

Implementa `learn_predictor(...)` con descenso por gradiente estocástico minimizando la pérdida *hinge*. Recorre los ejemplos de entrenamiento en orden, una vez por época, y actualiza los pesos con cada ejemplo. Entrena **solo** con `train_examples`: el conjunto de validación es únicamente para medir. Con `num_epochs=20` y `eta=0.01` sobre el corpus real debes conseguir menos de 4 % de error en entrenamiento y menos de 30 % en validación.

Pista

La actualización de los pesos es de una sola línea: `increment(weights, eta * y, phi)` cuando corresponda actualizar. La condición que decide *cuándo* corresponde sale del gradiente de la *hinge*: es cero cuando el margen supera 1, y vale $-\phi(x)y$ cuando no. Deja `verbose=True` mientras depuras para ver bajar el error época a época; el corrector automático lo apaga.

(c) n-gramas de caracteres [10 puntos]

Implementa `extract_character_features(n)`, que devuelve una *función* que mapea una cadena al conteo de todas sus subcadenas de largo n, después de eliminar los espacios en blanco (espacios y tabuladores).

```
>>> extract_character_features(3)("buena la peli")
{'bue': 1, 'uen': 1, 'ena': 1, 'nal': 1, 'ala': 1,
 'lap': 1, 'ape': 1, 'pel': 1, 'eli': 1}
```

Problema 2: Análisis del error [27 puntos]

Un modelo con 27 % de error de validación se equivoca en más de una de cada cuatro reseñas. La pregunta interesante no es cuánto se equivoca, sino **en qué** se equivoca.

(a) Las peores reseñas [10 puntos]

Implementa `error_analysis(examples, feature_extractor, weights, k)`, que devuelve las `k` reseñas peor clasificadas: solo las mal clasificadas, cada una como la tupla `(texto, etiqueta, predicción, margen)` con margen $= (\mathbf{w} \cdot \phi(x))\, y$, ordenadas de margen menor a mayor.

(b) Quién tuvo la culpa [8 puntos]

Implementa

`caracteristicas_culpables(x, y, feature_extractor, weights, k)`.

Para cada característica f de $\phi(x)$, con valor v y peso w_f, su aporte al margen es $v\, w_f\, y$. Devuelve las `k` características de aporte estrictamente negativo (las que empujaron el puntaje al lado contrario

de la etiqueta verdadera) como tuplas `(característica, aporte)`, de aporte menor a mayor y, ante empates, en orden alfabético.

Pista

Corre `python submission.py`: imprime las cinco peores reseñas de validación con sus culpables, y deja `pesos.txt` y `analisis-error.txt`, que trae el desglose `palabra: valor * peso = aporte` de cada reseña. Mirar esas dos salidas es la mejor forma de convencerse de qué está aprendiendo el modelo: si una palabra neutra como `feels` quedó con peso muy negativo, aprendió una correlación del corpus, no del idioma.

(c) Qué largo de n-grama conviene [4 puntos]

Implementa

```
mejor_n(train_examples, validation_examples, valores_de_n, num_epochs, eta),
```

que entrena un clasificador de n-gramas de caracteres por cada n pedido y devuelve la tupla `(mejor, errores)`: el diccionario `errores` lleva el error de validación de cada n, y `mejor` es el n de menor error (ante empate, el n más pequeño). Con `python submission.py ngramas` puedes ver el barrido completo de $n = 2, \ldots, 7$ sobre el corpus real.

(d) Una reseña hecha para los n-gramas [5 puntos]

Escribe en `P2_RESENA` (en la sección de respuestas de `submission.py`) una reseña **negativa** de al menos cuatro palabras tal que:

1. ninguna de sus palabras aparezca en el vocabulario del conjunto de entrenamiento, y
2. el clasificador de 6-gramas de caracteres, entrenado sobre el corpus real con `num_epochs=20` y `eta=0.01`, le dé puntaje negativo.

Una reseña así separa a los dos extractores: como el modelo de palabras no reconoce ninguna de sus palabras, su puntaje es exactamente 0 y predice $+1$, o sea se equivoca; el de caracteres, en cambio, acierta.

Pista

Deforma palabras que sí están en el corpus: superlativos y sufijos inventados, errores de tipeo, letras repetidas, palabras pegadas o con almohadilla. El 6-grama sobrevive a la deformación aunque la palabra completa no exista.

Problema 3: Margen suave, SVM y kernels [25 puntos]

Los Problemas 1 y 2 entrenan un clasificador lineal sin decir nada sobre *cuál* de todas las fronteras que separan bien los datos conviene elegir. La máquina de vectores de soporte (SVM) responde esa pregunta: elige la que deja el margen más ancho. Como el corpus real no es separable, se usa la versión de **margen suave**, que minimiza

$$J(\mathbf{w}, b) = \tfrac{1}{2}\|\mathbf{w}\|^2 + \frac{C}{n}\sum_{i=1}^{n} \max\{0,\ 1 - y_i(\mathbf{w}\cdot\mathbf{x}_i + b)\}.$$

El primer término premia los márgenes anchos (un $\|\mathbf{w}\|$ pequeño significa un margen $1/\|\mathbf{w}\|$ ancho) y el segundo castiga a los ejemplos que se meten dentro del margen o quedan mal clasificados. El parámetro C fija la **compensación** entre los dos: con C pequeño manda el margen ancho y el modelo tolera errores; con C grande manda el ajuste a los datos y el margen se angosta.

Llamamos **vectores de soporte** a los ejemplos con $y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \le 1$: los que están sobre el margen o adentro de él. Son los únicos que aportan al subgradiente, y por lo tanto los únicos que

determinan la frontera. Contarlos dice cuánto del conjunto de datos está efectivamente sosteniendo el modelo.

Un **kernel** es una función $K(\mathbf{a}, \mathbf{b})$ que devuelve el producto punto de las dos entradas *después* de transformarlas con un mapa ψ, sin construir nunca ψ:

$$K(\mathbf{a}, \mathbf{b}) = \psi(\mathbf{a}) \cdot \psi(\mathbf{b}).$$

Los dos que verás aquí son el polinomial $K(\mathbf{a}, \mathbf{b}) = (\mathbf{a} \cdot \mathbf{b} + c_0)^p$ y el gaussiano o RBF $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$. Para el polinomial de grado 2 con $c_0 = 1$ el mapa ψ sí se puede escribir, y en la parte (b) vas a comprobar la igualdad de arriba con números concretos. Ese es el *truco del kernel*: la frontera se vuelve curva en el espacio original sin que nadie pague el costo de trabajar en el espacio transformado.

La representación numérica de este problema

Los diccionarios dispersos de los Problemas 1 y 2 no sirven para `numpy.util.matriz_de_resenas(examples)` devuelve el par (X, y) con la misma información resumida en **dos** números por reseña:

$x_1 =$ cuántas palabras de la reseña están en el léxico positivo, $\qquad x_2 =$ ídem con el léxico negativo.

Los dos léxicos, de 600 palabras cada uno, se construyen contando etiquetas en el conjunto de *entrenamiento* con una regla determinista que está documentada en `util.py`. No son un diccionario de sentimiento hecho a mano: salen de los datos, y por eso incluyen palabras que a simple vista no dicen nada. Cada reseña queda así como un punto del plano, lo que permite dibujar la frontera y hace que todo el Problema 3 corra en menos de un segundo.

(a) Margen suave y vectores de soporte [11 puntos]

Implementa en `submission.py`, con `numpy` y sin bucles sobre los ejemplos:

- `entrenar_svm_margen_suave(X, y, C, epocas, eta)`, que minimiza $J(\mathbf{w}, b)$ por descenso por subgradiente sobre todo el conjunto a la vez. Con $v_i = 1$ si $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) < 1$ y $v_i = 0$ si no,

$$\frac{\partial J}{\partial \mathbf{w}} = \mathbf{w} - \frac{C}{n} \sum_i v_i \, y_i \, \mathbf{x}_i, \qquad \frac{\partial J}{\partial b} = -\frac{C}{n} \sum_i v_i \, y_i.$$

  Parte de $\mathbf{w} = \mathbf{0}$ y $b = 0$ y ejecuta exactamente `epocas` iteraciones, sin parada anticipada: así el resultado es reproducible.
- `indices_vectores_de_soporte(X, y, w, b, tol)`, que devuelve en orden creciente los índices con $y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \leq 1 + \texttt{tol}$.
- `error_de_clasificacion(X, y, w, b)`, la fracción de ejemplos mal clasificados, con la convención de siempre (puntaje 0 predice $+1$).

Pista

El paso η no se elige a ojo: `util.paso_seguro(X, C)` devuelve $1/(1 + C\,\overline{\|\mathbf{x}\|^2})$, que es el paso más grande con el que el descenso no oscila. Úsalo. Y verifica con `util.objetivo_svm(...)` que J baja iteración a iteración: si sube, tienes un signo invertido.

(b) Kernels y el mapa explícito [9 puntos]

Implementa `kernel_polinomial(a, b, grado, coef0)`, `kernel_rbf(a, b, gamma)` y `mapa_polinomial_grado2(x` Esta última devuelve el vector $\psi(\mathbf{x})$ del kernel polinomial de grado 2 con $c_0 = 1$, que para $\mathbf{x} \in \mathbb{R}^d$ tiene $(d+1)(d+2)/2$ coordenadas, en este orden:

$$1, \qquad \sqrt{2}\, x_i, \qquad x_i^2, \qquad \sqrt{2}\, x_i x_j \ (i < j).$$

El corrector verifica, con tolerancia relativa 10^{-9}, que $\psi(\mathbf{a}) \cdot \psi(\mathbf{b})$ coincide con $(\mathbf{a} \cdot \mathbf{b} + 1)^2$, además de la dimensión del vector y de las propiedades que todo kernel válido cumple (simetría, valor 1 en la diagonal del gaussiano y matriz de Gram semidefinida positiva).

(c) Los números del barrido de C [5 puntos]

Corre `python submission.py svm` y anota en la sección de respuestas de `submission.py`:

1. En `SVM_VECTORES_DE_SOPORTE`, `SVM_ERROR_ENTRENAMIENTO` y `SVM_ERROR_VALIDACION`, la tabla del barrido de $C \in \{0{,}1,\ 1,\ 10,\ 100\}$: cuántos vectores de soporte quedan y qué error de entrenamiento y de validación da cada valor. (3 puntos)

2. En `SVM_GRADO2_ERROR_VALIDACION` y `SGD_ERROR_VALIDACION`, el error de validación del SVM entrenado sobre el mapa polinomial de grado 2 (para esos mismos cuatro valores de C) y el del SGD sin regularización del Problema 1b entrenado sobre *las mismas* dos características. (2 puntos)

Los números se comparan con tres decimales, así que basta copiar los que imprime el programa. Las claves de cada diccionario son los cuatro valores de C, tal cual.

Pista

En la práctica nadie escribe esto a mano: `scikit-learn` trae `SVC(kernel='rbf', C=...)`, que además resuelve el problema dual y permite usar kernels sin mapa explícito. Aquí lo escribimos nosotros para que quede claro qué hace cada parámetro; en esta tarea `scikit-learn` no se usa ni para comparar.